

RELATÓRIO CONSOLIDADO

M.I.N.D. — Muñdji Intelligent Neural Developer
Quadro completo: o que existe, o que está provado, e o que falta

9	11 612	66	329	0
entregas de código	linhas	ficheiros	testes a passar	ciclos com LLM real

O último número é o mais importante deste relatório. O MIND está completo e verificado contra simulação, mas **nunca correu com um modelo de linguagem verdadeiro**. Tudo o que se segue deve ser lido com essa ressalva.

1. O que é o MIND

Sistema multi-agente autónomo de geração de código, sobre LangGraph, inspirado na estrutura do cérebro humano. Gera código através de um ciclo iterativo entre três papéis, até 98-100% de funcionalidade.

Papel	Componente	Responsabilidade
Orquestrador	CORTEX <i>único com persona (JARVIS)</i>	Cria a lógica e o código base, anota marcadores por secção, distribui aos NEURONS, valida contratos de interface, organiza o resultado, executa os testes e compila a cópia final aprovada.
Revisor	CEREBELLUM <i>sem persona</i>	Avalia o código base, audita o organizado, gera os testes pelos quais o CORTEX será avaliado, e decide a aprovação por validação cruzada de dois relatórios independentes.
Execução	NEURON 1-6 <i>sem persona</i>	Cada um recebe o código base completo mas implementa apenas a secção do seu marcador. Correm em paralelo, com circuit breaker individual.
Memória	HIPPOCAMPUS <i>opcional, desligado</i>	Camada de ML consultiva, treinada sobre o histórico local. Apoia auto-rejeição, nunca auto-aprovação.

Separação inegociável: o CORTEX não gera os testes pelos quais vai ser avaliado, e não decide sozinho a aprovação. O CEREBELLUM faz as duas coisas. É esta separação que impede o sistema de se avaliar a si próprio com critérios que ele mesmo escolheu.

2. Cronologia das entregas

Commit	Entrega	O que trouxe
ac25c16	Base do MIND	CORTEX, CEREBELLUM, 6 NEURONS, SYNAPSE DB (SQLite WAL), sandbox multi-linguagem, sanitizador, backup, versionamento git, CLI
fdcb706	HIPPOCAMPUS	Camada de ML consultiva com assimetria de segurança, cold-start seguro e promoção por comparação
4f51da6	Suite de testes	151 testes herméticos, um ficheiro por área
7c6d807	Fecho de lacunas	Diff real na validação de contrato, limpeza git a sério, unificação no LangGraph, esquema JSON nos relatórios
deec79d	HuggingFace / GPU alugada	Router com 4 modos, autenticação por componente, suporte a ids do Hub, adaptadores LoRA e checkpoints em disco
06a57a6	Sandbox evolutiva	Biblioteca de testes que cresce entre ciclos, rigor crescente em 3 níveis, percentagem medida em vez de estimada
a143abf	Preparação do piloto	Comandos <i>verificar</i> e <i>piloto</i> , tarefas de referência, script de aprovisionamento de vLLM
de9e0a1	Ensaio do runbook	Servidor de ensaio e validação das 4 passagens sem GPU; discrepância do <i>ml-status</i> corrigida

Mais 7 relatórios em PDF, um por etapa, todos versionados em *docs/relatorios/*. Este é o oitavo.

3. Decisões de arquitectura

As que moldaram o sistema, com a razão de cada uma. Todas estão documentadas no código ou em *docs/decisoess/*.

Decisão	Razão
Execução unificada no LangGraph	Havia duas implementações executáveis do ciclo. Um teste técnico verificou, em execução real, que o LangGraph exprime rollback, regra de divergência, activação dinâmica, circuit breaker e limite de iterações sem contorções. O orquestrador Python paralelo foi eliminado.
Diff real na validação de contrato	A heurística de marcadores é uma pista, não uma prova. O CORTEX fotografa as secções alheias antes de distribuir e compara-as depois — apanha um NEURON que mexa no preâmbulo partilhado sem tocar em marcador nenhum.
Percentagem medida, não estimada	Com a sandbox evolutiva, a percentagem vem dos testes que passaram: nível 1 = 33%, nível 2 = 66%, nível 3 = 99%, e 1% para conformidade de formato. Um nível sem testes contribui 0, e por isso não pode aprovar.
Assimetria de segurança no ML	O HIPPOCAMPUS pode apoiar auto-rejeição, nunca auto-aprovação. Um modelo imaturo a rejeitar custa um ciclo; a aprovar, custa a garantia toda.
Esquema JSON com recurso ao regex	O parsing por expressão regular sobre texto livre é frágil com modelos reais. O JSON é o formato pedido; o regex fica como último recurso e cada desvio é registado — é o que vai dizer, no piloto, que modelos cumprem.
openai_compat por omissão	Com GPU alugada, o servidor (vLLM/TGI) gere a memória e o batching, e o MIND só fala HTTP. Adaptadores LoRA são servidos como modelos distintos, o que torna 8 componentes viáveis numa só GPU.

4. Verificação — 329 testes

Área	N.º	Área	N.º
Sandbox evolutiva	70	Marcadores	17
Router de modelos	31	Sandbox multi-linguagem	16
Sanitizador	27	Pipeline de ML	13
Piloto	25	Limpeza git	13
HIPPOCAMPUS	24	Activação dinâmica	9
Esquema de relatórios	23	Regressão da base	7
Contratos de interface	20		
SYNAPSE DB	20		
Ciclo ponta-a-ponta	18		

Os testes são herméticos: cada um recebe base de dados e workspace próprios em directório temporário, e uma fixture automática isola as variáveis de ambiente. Correm com um único *pytest*, em cerca de 16 segundos. Os 4 saltados são por toolchains ausentes no ambiente (Rust), não por falha.

4.1 O ensaio do runbook

Além da suite, a sequência operacional do piloto foi ensaiada contra um servidor falso: as 4 passagens correm de ponta a ponta, a sandbox evolutiva gera e executa testes nos 3 níveis, e a herança entre ciclos foi observada — na segunda corrida das mesmas tarefas, cada uma herdou 3 testes dos ciclos aprovados.

5. O que está provado — e o que não está

É a distinção mais importante deste relatório.

Provado (com simulação)	Por provar (exige modelo real)
O ciclo das 3 fases corre de ponta a ponta e aprova	Se um modelo real produz código que chega a 98%
Os contratos de interface apanham violações reais	Se os modelos respeitam os marcadores na prática
A sandbox executa Python, Go e C de verdade	Quanto demora um ciclo, e quantas iterações precisa
O sanitizador substitui, comenta e é idempotente	Se os modelos respeitam o esquema JSON (e em que percentagem)
A limpeza git remove objectos fisicamente	Se NEURON_TIMEOUT_SECONDS=60 e SANDBOX_TIMEOUT=30 são realistas
A biblioteca de testes cresce e é herdada entre ciclos	Se os testes gerados por um modelo real são executáveis
A percentagem é calculada sobre resultados reais	Que modelo escolher para cada componente
O HIPPOCAMPUS degrada em segurança sem dados	Se o histórico chega a ter volume para o treinar

6. Lacunas conhecidas e decisões de não-fazer

6.1 Decisões conscientes de NÃO fazer

Documentadas no código como decisões tomadas, não como lacunas por resolver:

Item	Razão	Caminho futuro
Isolamento do sandbox por contentores	O MIND corre o seu próprio código, não código hostil de terceiros. Complexidade prematura.	Contentores efémeros ou namespaces, se algum dia correr código não confiável
Saída em ficheiro único	Impacto baixo enquanto as tarefas couberem num ficheiro.	Marcadores a transportarem o destino: <code>[NEURON_2:python:auth/x.py]</code>
Redis	Com um só servidor de inferência, uma fila não traz paralelismo real.	Vários servidores ou GPUs
Ranking por qualidade histórica nos testes	Exige dados que ainda não existem. As contagens já são alimentadas.	Quando o HIPPOCAMPUS tiver histórico suficiente
GUI	Fora do escopo por indicação da especificação.	Decisão futura; a separação lógica/interface já é limpa

6.2 Defeitos encontrados e corrigidos ao longo do trabalho

Registados aqui porque nenhum tinha sintoma visível — todos foram descobertos por escrever testes ou ensaiar procedimentos, não por falharem:

- **O versionamento git era decorativo.** O MIND nunca escrevia o código no workspace durante as iterações, só na aprovação. O commit por iteração não tinha conteúdo, e o rollback e a intervenção manual apoiavam-se num histórico vazio.
- **As falhas de chamada a modelo nunca chegavam à base de dados.** Eram registadas com `cycle_id=0`, que viola a foreign key; o insert falhava e o except engolia o erro.
- **A limpeza git não podava nada.** As tags antigas mantinham os commits originais alcançáveis e, através dos seus pais, também os temporários.
- **O runbook apontava para o sítio errado.** Mandava verificar a conformidade JSON com `ml-status`, que não a reportava.

7. O que falta: o piloto

Uma coisa só, e não depende de mais código. **Três valores:** `MODEL_ENDPOINT`, `MODEL_AUTH_TOKEN` e o id do modelo.

Passagem	O que faz	O que produz
1 — ciclo base (sandbox desligada)	verificar, depois <code>piloto --max-tarefas 3</code> , depois aferir os timeouts com valores medidos, depois as 10 tarefas	As primeiras medições verdadeiras: duração, iterações, taxa de aprovação, conformidade JSON
2 — critério de selecção	Preencher <code>model_selection_criteria.md</code> com o CSV do piloto	Decisão por componente: manter, substituir, ou marcar para fine-tuning. A decisão é tua — o CSV dá os números, não o juízo
3 — sandbox evolutiva	Repetir as 10 tarefas com <code>SANDBOX_TESTS_ENABLED=true</code>	Quantos testes por nível, que percentagem é executável, se a herança funciona, e quanto a percentagem medida difere da estimada
4 — HIPPOCAMPUS	Quando <code>ml-status</code> confirmar amostras suficientes	Modelo promovido ou não, e a métrica de validação

Princípios que vou cumprir durante o piloto: se algo correr mal, reporto com os logs antes de corrigir — não ajusto parâmetros às cegas para «fazer passar», porque isso falsifica o resultado. Uma tarefa que falha não interrompe as seguintes. E confirmo que o backup local está feito antes de dizer que o custo pode parar.

Relatório consolidado das 9 entregas de código. Todas as verificações descritas foram executadas; nenhuma envolveu um modelo de linguagem real. Os relatórios por etapa estão em *mind/docs/relatorios/*, indexados no README.